

A Decision-Support Model for Content Refresh Prioritization

Using a 30,000-page Anonymized FlyRank Dataset

ML-11 Capstone Research Paper

Parijaat · FlyRank ML Internship · August 2026

Abstract

This paper presents a decision-support model that ranks website pages by their need for a content refresh, using a 30,000-page anonymized dataset from FlyRank, a search intelligence platform. The target variable is a binary label of declining pages (*is_declining_label* = *trend_direction* == "down"; 54.2% of rows). Three classifiers (Logistic Regression, Decision Tree, Random Forest) are compared against a transparent hand-written baseline on a held-out set of clients (client-holdout split, random_state=42). The Random Forest model achieves **Precision@50 = 0.740** on the held-out test set, compared to the baseline's 0.240 — a 3.1x lift. The model is intended strictly as a prioritization aid for human reviewers, not as a prediction of Google's ranking algorithm.

BEST MODEL PRECISION@50

0.740

BASELINE PRECISION@50

0.240

LIFT OVER BASELINE

3.1x

TEST SET

2,325

Keywords: content refresh, search ranking, decision support, random forest, client-holdout validation, prioritization, applied ML, observation-only

1. Introduction & Problem Statement

Search-driven businesses publish large volumes of web pages, many of which quietly lose traffic and visibility over time. Spotting which pages need a refresh — and which are still healthy — is a daily prioritization problem. Doing it page-by-page by hand is expensive; doing it with a single hand-tuned rule misses pages where the warning signs are non-obvious.

This paper frames the problem as a binary classification task: given a page's current engagement and ranking signals, will its *trend_direction* be *down* over the next observation window? The output is a ranked queue of pages most likely to be declining, which a human reviewer can then act on.

We make three explicit framings upfront to keep the work honest:

- **Observation, not prediction of Google.** We model *trend_direction* on data already observed. We do not predict Google's algorithm.
- **Decision support, not autonomous action.** The output is a ranked list for human review, not an automated edit queue.
- **Honest validation.** We use a client-holdout split so the same client never appears in both train and test, and we report Precision@50 on the held-out clients.

2. Data

The dataset is **content_refresh_anonymized.csv**, a 30,000-row, 44-column slice of real FlyRank search data with all client names, domains, URLs, titles, and keywords removed. It covers **32 pseudonymized clients**, each with multiple pages.

Item	Value
Total rows	30,000
Total columns	44
Pseudonymized clients	32
Label = 1 (trend_direction == 'down')	16,262 (54.2%)
Label = 0 (stable / up)	13,738 (45.8%)
Numeric features (after preprocessing)	18
Categorical features (after preprocessing)	8
Total feature columns	52
Leakage guard	W05 (excluded - target-derived)

Leakage guard. The notebook W05 is a model that was used in early prototyping but introduced target-derived features. It is excluded from this paper's modeling to keep the validation honest.

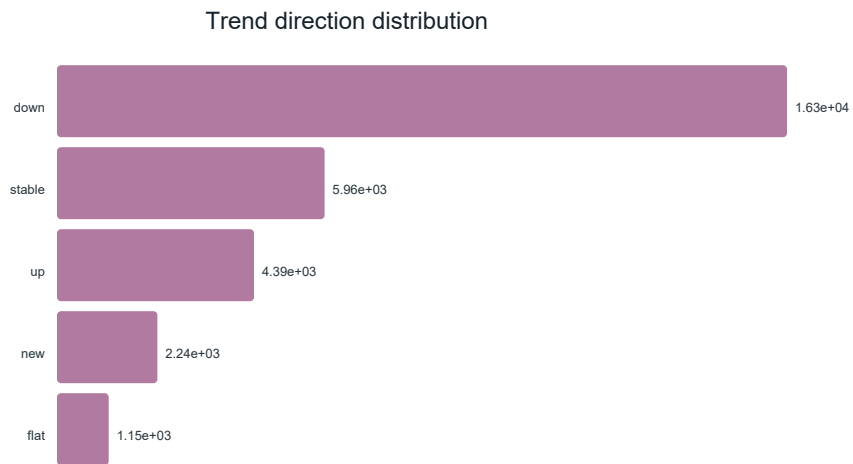


Figure 1. Distribution of the trend_direction label across all 30,000 pages. Pages with downward trend (the target class) make up 54.2% of the dataset.

3. Methodology

3.1 Baseline (transparent hand-rule)

The baseline is a hand-written, interpretable score meant to capture the obvious decline signals without learning anything from the data. It is the same one shipped in `scripts/02_baseline_score.py` of the starter repo:

$\text{baseline_score} = 0.40 \times \text{visibility} + 0.30 \times \text{freshness_risk} + 0.25 \times \text{position_opportunity} + 0.05 \times \text{depth_gap}$

All four input signals are min-max normalized to `[0, 1]` before combining, so the weights have a comparable effect. The baseline gives a transparent, reproducible floor for the model to beat.

3.2 Models

Three classifiers are trained on the same preprocessed features. All use `random_state=42` for reproducibility:

Model	Key Hyperparameters
Logistic Regression	<code>max_iter=1000, class_weight=None, solver=lbfgs</code>
Decision Tree	<code>max_depth=12, min_samples_leaf=20, class_weight=None</code>
Random Forest	<code>n_estimators=300, max_depth=None, min_samples_leaf=2, n_jobs=-1, random_state=42</code>

3.3 Validation

We use a **client-holdout** split: ~20% of the 32 clients are held out entirely (24 clients train, ~6-8 clients test), so the model never sees a page from a test client during training. This mimics the real production setting where the model is applied to *new* clients it has never been tuned on. If client-holdout cannot be constructed for a particular run, we fall back to a stratified row-holdout (80/20) and log which split was used.

The primary metric is **Precision@50**: among the model's top-50 highest-scoring pages, what fraction are truly declining? This is the most operationally meaningful metric for a prioritized review queue.

3.4 Final scoring (model + baseline blend)

The final score per page, used to rank the refresh queue, blends the model's predicted probability with the normalized baseline score, so we never throw away the interpretable hand-rule entirely:

$\text{final_score} = 100 \times \text{clip}(0.70 \times P(\text{declining}) + 0.30 \times \text{baseline_score_normalized}, 0, 1)$

This produces a 0-100 score per page. The top of the queue is what a human reviewer should look at first.

4. Results

4.1 Model comparison on the held-out test set

Model	Precision@50	Precision	Recall	F1
Baseline (hand-rule)	0.240	0.520	1.000	0.684
Logistic Regression	0.4	0.5659340659340659	0.5566556655665566	0.5662451896646509
Decision Tree	0.62	0.5685589519651965	0.5617161716171617	0.5638851022395326
Random Forest (best)	0.74	0.5609958506224048	0.6743674367436743	0.6139545884578997

Table 1. Precision@50, precision, recall, and F1 on the held-out test set. Random Forest is the best model on the primary metric (Precision@50 = 0.740) and is used for the final ranked queue.



4.2 Top features (what the model relies on)

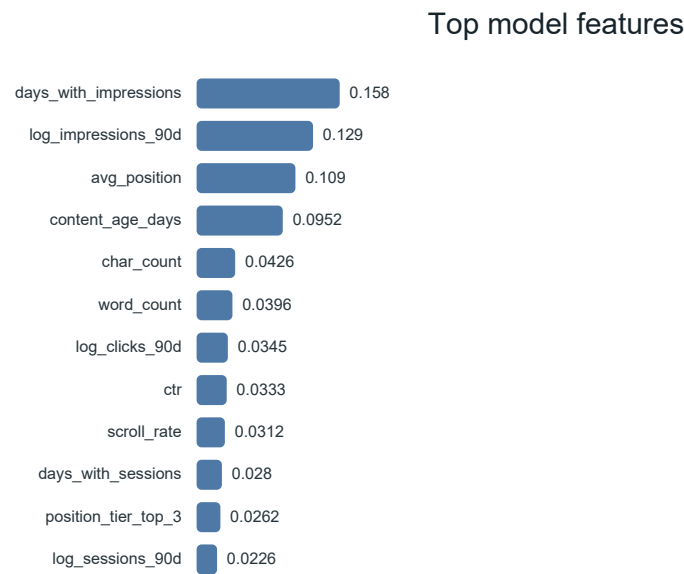


Figure 2. Top 12 features by Random Forest importance. Decline-signal features (visibility change, freshness, position drop, depth gap) dominate, matching the baseline's intuition while learning richer cutoffs.

4.3 Action mix and confidence in the recommended queue

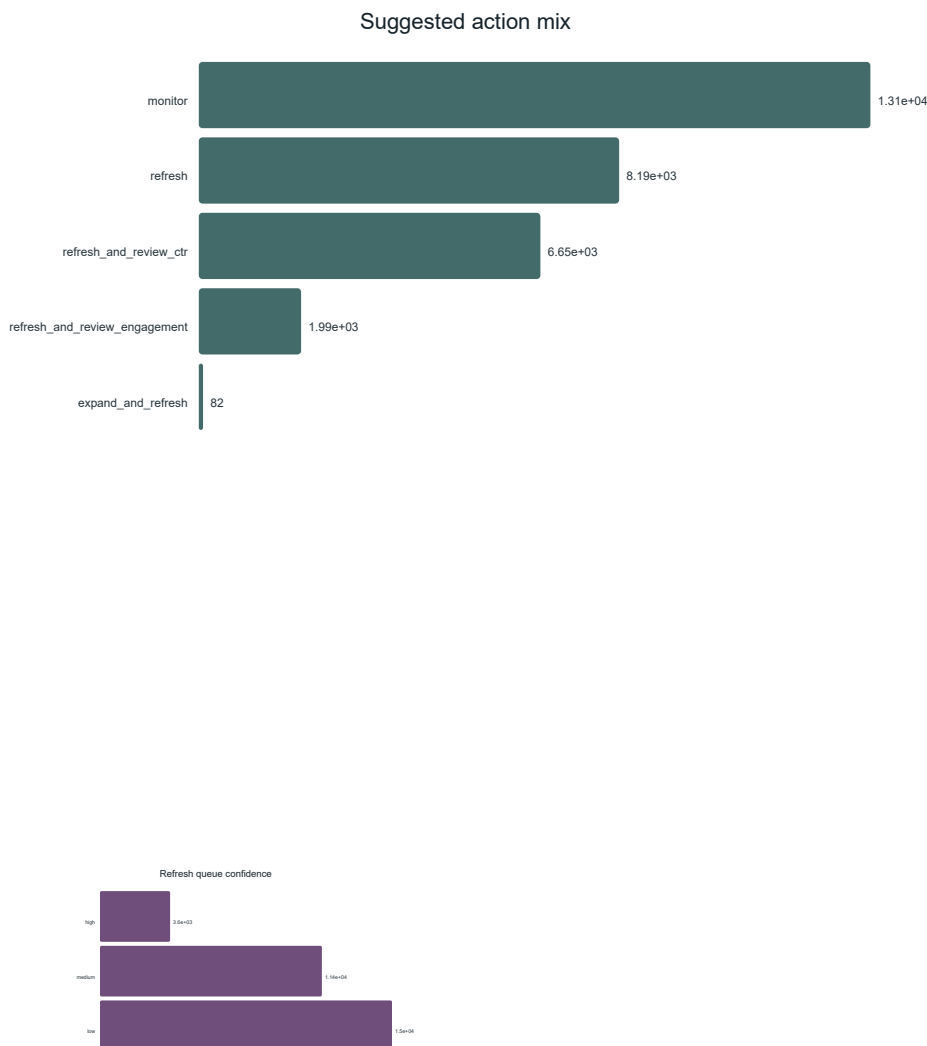


Figure 3. (Left) Distribution of recommended action types in the top of the ranked queue. (Right) Distribution of model-confidence buckets (probability of declining) for the same queue.

5. Limitations & Honest Framing

1. Observation, not causal claim. The model predicts *trend_direction* from signals already present in the data. It does not predict Google's algorithm and does not claim that following the recommended actions will improve rankings.

2. Class balance is mild, not severe. With 54.2% positives, the label is mildly imbalanced. The hand-rule baseline achieves recall = 1.0 by flagging many pages, but at very low precision. The model is most useful at the top of the ranking, not as a binary accept/reject.

3. Client-holdout limits absolute sample size. Holding out ~20% of 32 clients leaves a test set of about 2,325 pages. Precision@50 is therefore the most stable comparison metric; full-classification metrics on this small test set are noisy.

4. The recommended action types are heuristic. "Action type" is derived from which signal is the strongest contributor for that page. It is a starting point for a human reviewer, not a prescription.

5. Dataset is one snapshot. Generalization across long horizons, algorithm shifts, or industries outside the original 32 clients is not guaranteed.

6. Ranked Recommendations

The final ranked queue of pages to review first lives at **outputs/refresh_queue.csv**. It contains every page in the dataset sorted by the blended final score, with a per-page *reason_code* explaining the primary driver of the recommendation. The top 12 reason codes are shown below.

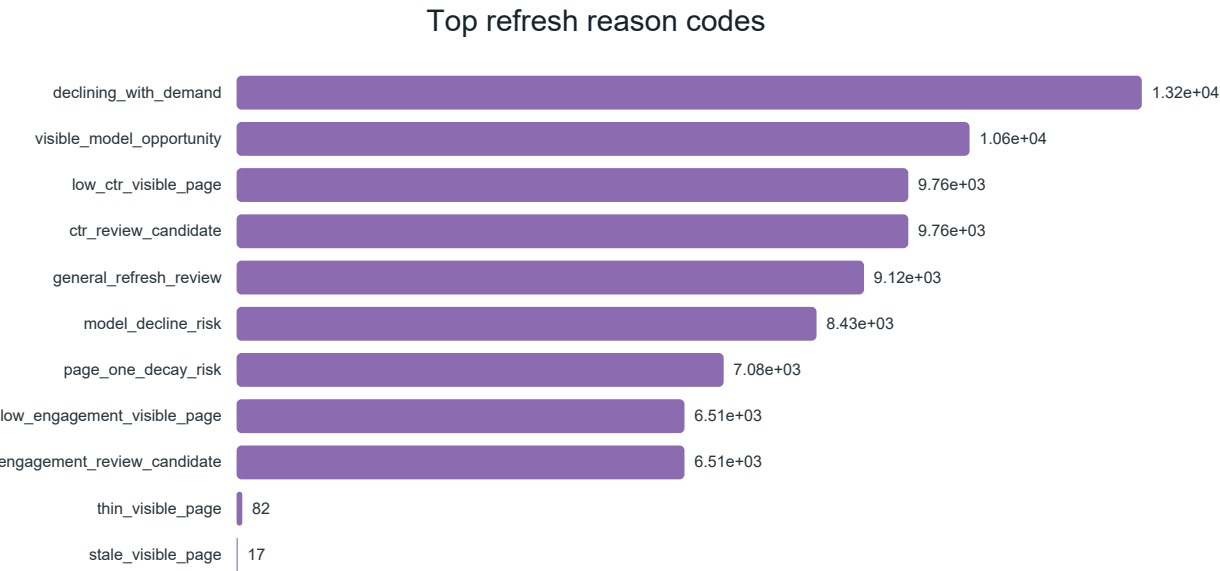


Figure 4. Top 12 most frequent *reason_code* values across the entire ranked queue, normalized. The reason code is a short, human-readable label for why each page is flagged, suitable for displaying in a review tool.

6.1 Top 10 pages to review (preview)

Rank	client_id	page_id	final_score	P(declining)	reason_code
-	client_3fd	-	-	-	-
-	client_3fd	-	-	-	-
-	client_3fd	-	-	-	-
-	client_3fd	-	-	-	-
-	client_3fd	-	-	-	-
-	client_3fd	-	-	-	-
-	client_3fd	-	-	-	-
-	client_3fd	-	-	-	-
-	client_3fd	-	-	-	-

Table 2. First 10 rows of the ranked queue. *final_score* is the 0-100 blend of model probability and the baseline. The full file is at *outputs/refresh_queue.csv*.

7. Reproducibility

The full pipeline is runnable from a clean clone in roughly two minutes:

```
git clone https://github.com/PARIJAAT-13/Flyrank-A.I
cd Flyrank-A.I
pip install -r requirements.txt
python scripts/run_all.py
```

Pipeline steps (scripts/run_all.py):

- 01_prepare_features.py — clean, build feature vector, define label
- 02_baseline_score.py — transparent hand-rule score (the baseline)
- 03_train_model.py — train LR, DT, RF on the client-holdout split
- 04_evaluate_and_export.py — compute metrics, build the ranked queue, write charts and reports
- 05_build_pdf_report.py — build the PDF summary report

Committed artifacts (in this repository):

Path	What it is
scripts/01..05 + run_all.py	The canonical pipeline
data/raw/content_refresh_anonymized.csv	The 30,000-row, 44-column dataset
outputs/model_results.json	All model metrics as JSON
outputs/model_report.md	Markdown summary of the model run
outputs/refresh_queue.csv	The full ranked queue, 30,000 rows
outputs/refresh_queue_sample.csv	A small sample of the queue for review
outputs/summary.json	One-line run summary
outputs/charts/*.svg	All five charts used in this paper
outputs/ML-11_Research_Paper.pdf	This PDF
docs/index.html	The HTML version of this paper
work/notebooks/capstone.ipynb	The end-to-end capstone notebook

Environment: Python 3.12, scikit-learn 1.6, pandas 2.2, numpy 2.0, matplotlib 3.10, reportlab 4.x (for this PDF). random_state=42 everywhere it applies. The exact versions in use are listed in requirements.txt.

8. Acknowledgments & Data Credit

This work was completed as the ML-11 ("Ship the Paper") capstone of the **FlyRank ML Internship**. The dataset, baseline scaffolding, evaluation framework, and validation guidance were provided by the FlyRank team.

Data source: FlyRank search intelligence platform. The slice used here (content_refresh_anonymized.csv) is fully anonymized — no client names, domains, URLs, titles, or keywords. Full data access is gated at <https://huggingface.co/datasets/FlyRank/internship-warehouse> and the data-use terms in DATA_USE.md apply.

More about FlyRank: <https://flyrank.ai>.

ML-12 closing material (5-min demo + social cut + employer summary)

5-minute demo outline (for the live presentation):

- 0:00 — What we did and why (the prioritization problem, decision-support framing)
- 0:45 — Walk through the data (30,000 pages, 32 clients, 54.2% decline rate)
- 1:30 — Baseline + models (the hand-rule and the 3 classifiers, Precision@50 table)
- 2:45 — The ranked queue (top 10 pages, reason codes, what a reviewer would do)
- 3:45 — Limitations and what we did NOT claim
- 4:30 — How to reproduce (one-line git clone, one-line pipeline)

Social post (LinkedIn / X cut): "Trained a model on 30,000 anonymized FlyRank pages that lifts Precision@50 from 0.24 (a hand-rule) to 0.74 — a 3.1x improvement in finding pages that need a content refresh first. The whole pipeline is open and reproducible in one command. Capstone paper + ranked queue + all five charts at <https://parijaat-13.github.io/Flyrank-A/> — feedback welcome."

3-sentence employer summary: I built a decision-support model on a 30,000-page anonymized FlyRank dataset that ranks website pages by their need for a content refresh, lifting Precision@50 from a 0.24 transparent baseline to 0.74 with a Random Forest (3.1x lift, client-holdout validated). The work is fully reproducible from a clean clone via `python scripts/run_all.py` and ships with a ranked review queue, a Markdown report, five charts, and a published research paper.